GLOBAL SOLUTION

Disciplina: Cloud

Trabalho: API

CyberIncident API

Plataforma de Gerenciamento de Incidentes de Segurança Cibernética

Aluno: Breno Aveiro

Data: 02/11/2026

Sumário

1. Introdução
2. Objetivos
3. Tecnologias utilizadas
4. Arquitetura
5. API REST
6. Banco de dados SQLite
7. Microsoft Azure
8. Implantação
9. Segurança
10. Testes
11. Evidências
12. Diagramas
13. Custos
14. Limitações e melhorias possíveis
15. Conclusão
16. Referências

1. Introdução

Toda organização que usa sistemas de informação enfrenta incidentes de segurança: tentativas de acesso sem autorização, ataques de força bruta contra senhas, mensagens de phishing, programas maliciosos, tráfego de rede incomum e possíveis vazamentos de dados. Para que um incidente seja tratado, ele precisa ser registrado, classificado por gravidade e acompanhado até a resolução. Em empresas maiores, essa tarefa fica a cargo de uma central de operações de segurança, conhecida pela sigla SOC (*Security Operations Center*).

Este trabalho apresenta a **CyberIncident API**, uma aplicação acadêmica que simula o núcleo do sistema de registro de um SOC. A aplicação não monitora redes nem detecta ameaças: ela permite cadastrar, consultar, classificar e acompanhar incidentes **fictícios**, e serve para demonstrar, de ponta a ponta, o desenvolvimento e a hospedagem de uma API REST na nuvem. O código está publicado em https://github.com/brenoaveiro/api-tarefas-fastapi-sqlite-azure e a aplicação responde em https://20.80.10.214/.

O projeto começou como uma API de tarefas, publicada anteriormente na mesma infraestrutura. A máquina virtual, o disco e o banco foram preservados, e o domínio da aplicação, o modelo de dados, os endpoints, a interface e a documentação foram substituídos pelo tema de segurança cibernética. A tabela antiga do banco continua no arquivo, sem alterações.

2. Objetivos

O objetivo geral é desenvolver, testar e publicar na Microsoft Azure uma API REST com persistência em SQLite, documentada e demonstrável pela internet. Os objetivos específicos são:

- modelar incidentes de segurança com categoria, severidade, status, origem e ativo afetado;
- oferecer criação, consulta, atualização e exclusão, além de estatísticas calculadas a partir dos dados reais;
- construir um painel (dashboard) e uma documentação interativa com identidade visual própria;
- hospedar a aplicação em uma máquina virtual Azure, com Nginx, systemd e disco persistente;
- proteger o acesso com firewall, restrição de escrita, HTTPS e boas práticas de programação segura;
- validar o funcionamento com testes automatizados locais, testes na URL pública e integração contínua;
- registrar evidências reais e documentar decisões técnicas, custos e limitações.

3. Tecnologias utilizadas

Python é a linguagem de programação da aplicação (versão 3.12 na máquina virtual). Foi escolhida pela legibilidade, pelo ecossistema para desenvolvimento web e pela facilidade de testar o código. Toda a lógica de negócio, a validação e o acesso ao banco estão escritos em Python.

FastAPI é o framework web. Ele recebe as requisições HTTP, associa cada rota a uma função Python, valida os dados e gera automaticamente a documentação OpenAPI. Foi preferido a alternativas como Flask porque a validação e a documentação nascem das mesmas definições de tipos, o que reduz erros e repetição de código.

Pydantic valida os dados de entrada e define o formato dos dados de saída. É nele que se declara, por exemplo, que a severidade só pode ser baixa, média, alta ou crítica, e que o endereço IP precisa ser válido. Dados inválidos são rejeitados antes de chegarem ao banco, com resposta HTTP 422.

SQLAlchemy é o ORM (*Object-Relational Mapper*): converte objetos Python em comandos SQL e o inverso. Como todas as consultas são montadas com parâmetros, e nunca por concatenação de texto, a aplicação não é vulnerável a injeção de SQL.

SQLite é o banco de dados relacional. Todo o conteúdo (tabelas, índices e registros) fica em um único arquivo, sem necessidade de um servidor de banco separado.

Uvicorn é o servidor ASGI que executa a aplicação FastAPI: abre uma porta, interpreta o protocolo HTTP e chama o código Python. Na máquina virtual, ele escuta somente em 127.0.0.1:8000.

Nginx é o servidor web que fica na frente do Uvicorn. É o único componente que recebe tráfego da internet (portas 80 e 443); entrega as requisições à aplicação (proxy reverso), termina o HTTPS e aplica a regra de restrição de escrita por endereço IP.

systemd é o gerenciador de serviços do Linux. Ele inicia a API quando a máquina liga, a reinicia se ela falhar e a isola do restante do sistema por meio de restrições de segurança.

HTML, CSS e JavaScript formam o dashboard e o tema do Swagger. Foram usados sem frameworks, para manter o consumo de memória baixo em uma máquina com 1 GiB de RAM e para evitar dependências externas.

Swagger UI é a interface de documentação interativa, servida a partir de arquivos locais do próprio projeto (sem CDN externo), com tema escuro personalizado. **Git e GitHub** guardam o histórico do código e executam a integração contínua (GitHub Actions). **Pytest** executa os testes automatizados e **Mermaid** desenha os diagramas.

As versões fixadas em requirements.txt são FastAPI 0.162.0, Starlette 1.10.0, Pydantic 2.14.2, SQLAlchemy 2.0.55 e Uvicorn 0.54.0. Uma auditoria com a ferramenta pip-audit apontou 14 vulnerabilidades conhecidas na versão anterior do Starlette (0.41.3); a atualização as eliminou, e a auditoria atual não encontra vulnerabilidades conhecidas nas dependências.

4. Arquitetura

A requisição de um usuário percorre estes componentes: navegador, IP público, grupo de segurança de rede (NSG), rede virtual, Nginx, Uvicorn/FastAPI e SQLite. O Nginx recebe a conexão, que pode ser HTTPS (porta 443) ou HTTP (porta 80), e repassa o pedido ao Uvicorn pela interface local. O FastAPI valida os dados, executa a regra de negócio e consulta o SQLite por meio do SQLAlchemy; a resposta volta pelo mesmo caminho.

Essa separação em camadas tem motivos práticos. O Uvicorn nunca fica exposto diretamente à internet, o que reduz a superfície de ataque. O Nginx concentra o HTTPS e a política de acesso. O arquivo SQLite fica no disco gerenciado da máquina virtual, que

persiste mesmo se a máquina for reiniciada. Os diagramas 1 e 2 (seção 12) mostram a estrutura e o fluxo.

5. API REST

Uma API REST expõe recursos (aqui, os incidentes) por meio de endereços (rotas) e dos métodos do protocolo HTTP, trocando dados em formato JSON. O método indica a intenção: GET consulta, POST cria, PUT substitui os dados de um recurso, PATCH altera parte dele e DELETE remove. O código de resposta informa o resultado: 200 (sucesso), 201 (criado), 204 (sem conteúdo, usado na exclusão), 404 (recurso inexistente), 422 (dados inválidos) e 500 (erro interno).

Método e rota	**Função**	**Códigos**
GET /	Dashboard (página HTML)	200
GET /health	Verifica a API e a comunicação com o SQLite	200, 503
GET /incidentes	Lista com paginação e filtros por severidade, status e categoria	200, 422
GET /incidentes/estatisticas	Contagens reais por status, severidade e categoria	200
GET /incidentes/{id}	Consulta um incidente	200, 404
POST /incidentes	Cadastra um incidente (status inicial: aberto)	201, 422
PUT /incidentes/{id}	Atualiza todos os campos	200, 404, 422
PATCH /incidentes/{id}/status	Altera somente o status	200, 404, 422
DELETE /incidentes/{id}	Exclui um incidente de demonstração	204, 404

Validações. Título e descrição têm tamanho mínimo e máximo; categoria e severidade só aceitam os valores permitidos; o endereço IP de origem, quando informado, é verificado com o módulo ipaddress do Python (IPv4 ou IPv6); campos opcionais vazios são tratados como ausentes. Os mesmos valores são garantidos de novo no banco, por restrições CHECK.

Regras de negócio. O status segue um fluxo de tratamento: aberto, em análise, contido e resolvido (Diagrama 4). Passar direto de *aberto* para *resolvido*, por exemplo, é rejeitado com HTTP 422. Ao resolver um incidente, o campo resolvido_em recebe a data e a hora em UTC; ao reabri-lo, o campo é limpo.

Estatísticas. A rota /incidentes/estatisticas executa consultas GROUP BY no SQLite e devolve as contagens por status, severidade e categoria, incluindo zero para os valores sem registros. Ela é declarada antes da rota de consulta por identificador, para não ser confundida com um ID. O dashboard consome essa mesma rota, de modo que todos os números da tela vêm do banco.

Erros. Falhas internas retornam HTTP 500 com mensagem genérica, sem expor detalhes técnicos, e a falha do banco no health check devolve 503.

Dashboard e Swagger. O dashboard (GET /) tem tema escuro de estilo SOC, indicador de disponibilidade, quatro cartões (total, críticos, abertos e resolvidos), gráfico de barras por severidade e tabela dos incidentes mais recentes, com layout responsivo. Os dados são inseridos na página com textContent, nunca como HTML, o que impede a execução de

código injetado (XSS); uma política de segurança de conteúdo (CSP) restringe ainda os recursos que o navegador pode carregar. A documentação em /docs é o Swagger UI com arquivos locais, tema próprio, descrições em português e seções por tipo de operação. Ela continua interativa e permite executar as requisições.

6. Banco de dados SQLite

O SQLite é um banco relacional que guarda tudo em um único arquivo: /var/lib/api-tarefas/tarefas.db. A aplicação usa duas tabelas: incidentes, da versão atual, e tarefas, da versão anterior, que foi preservada sem alterações. A tabela incidentes tem os campos id (chave primária), titulo, descricao, categoria, severidade, status, ip_origem, ativo_afetado, responsavel, criado_em, atualizado_em e resolvido_em. As datas são gravadas em UTC. Há índices em id, categoria, severidade e status, que são os campos usados em filtros e agrupamentos, e restrições CHECK que impedem valores fora das listas permitidas, mesmo que alguém grave no banco sem passar pela API (Diagrama 3).

O banco opera em modo WAL (*write-ahead logging*): as escritas vão primeiro para um arquivo de registro, o que reduz o bloqueio entre leituras e escritas. O arquivo pertence ao usuário de serviço apitarefas e tem permissão 600, isto é, só esse usuário o lê e escreve.

Persistência. O arquivo do banco está no disco gerenciado da máquina virtual. Reiniciar o serviço, ou a própria máquina, não apaga os dados, pois o disco é um recurso independente do processo da aplicação. Isso foi comprovado reiniciando o serviço e consultando o mesmo registro (Figura 16).

Migração e backup. Ao iniciar, a aplicação cria apenas as tabelas que ainda não existem, de forma repetível e sem apagar dados. Antes de cada atualização, o script de implantação gera um backup consistente com a API de backup do SQLite (.backup, que é seguro com o banco em uso) e verifica a integridade da cópia; se o backup falhar, a atualização é abortada. A existência das tabelas é verificada pelo catálogo sqlite_master, e não pela quantidade de linhas.

Por que SQLite e quais os limites. Ele não exige servidor de banco, não gera custo extra e atende ao volume deste trabalho. Em contrapartida, não oferece replicação e lida mal com muitas escritas simultâneas; em um sistema real de maior escala, seria preferível um serviço gerenciado, como o Azure SQL Database.

7. Microsoft Azure

O projeto usa uma assinatura **Azure for Students**, que concede créditos e tem políticas próprias: só permite criar recursos em algumas regiões e limita as famílias de máquinas virtuais com cota. Todos os recursos estão no grupo de recursos GS_Cloud_API, na região North Central US, e foram identificados com o padrão de nomes da prova.

Resource Group. É um contêiner lógico que agrupa os recursos de uma mesma solução. Facilita organizar, aplicar permissões e remover tudo de uma vez: apagar o grupo encerra a cobrança de todos os itens.

Máquina virtual e SKU. A máquina GS-Cloud-API-VM é um computador virtual Linux (Ubuntu Server 24.04 LTS, versão de suporte longo). O tamanho (SKU) define seus recursos e preço. A Standard_B2ats_v2 pertence à série B, chamada *burstable*: tem desempenho de CPU base baixo e acumula créditos nos períodos ociosos, que são gastos

em picos, o que é adequado para uma API de baixo tráfego. Tem **2 vCPUs** (processadores virtuais, que executam as instruções do programa) e **1 GiB de RAM** (memória volátil, em que ficam os processos em execução; é muito mais rápida que o disco, mas se perde ao desligar). Pelo padrão de nomes da Azure, o sufixo indica processador AMD e memória reduzida. Como 1 GiB é pouco para atualizar pacotes do sistema, foi criado um arquivo de troca (*swap*) de 1 GiB no disco.

Por que não a B1s. A primeira opção planejada era a Standard_B1s, de menor preço. Ao consultar a assinatura, constatou-se que ela não está disponível; que uma política limita as regiões a South Africa North, North Central US, Brazil South, East US 2 e Mexico Central; e que as máquinas liberadas em Brazil South tinham cota zero. A Standard_B2ats_v2 em North Central US foi a opção de menor custo permitida e com cota.

Disco gerenciado. O GS_Cloud_API_Disk é um *Managed Disk* do tipo Standard SSD LRS, com 32 GiB. Disco gerenciado é um disco virtual cujo armazenamento a Azure administra, independente da máquina. *Standard SSD* indica a classe de desempenho (o portal exibe 500 IOPS e 100 MB/s de vazão máxima) e *LRS* (armazenamento localmente redundante) mantém três cópias dos dados no mesmo datacenter. É nele que ficam o sistema, o código e o SQLite.

Rede. A rede virtual (VNet) GS_Cloud_API_VNet (10.10.0.0/16) é uma rede privada isolada, dividida em sub-redes; a sub-rede GS_Cloud_API_Subnet (10.10.1.0/24) abriga a máquina, que recebe o endereço privado 10.10.1.4 por meio de uma interface de rede (NIC). O **IP público** GS_Cloud_API_IP (SKU Standard, alocação estática) é o endereço fixo pelo qual a internet chega à máquina: 20.80.10.214.

NSG. O grupo de segurança de rede GS_Cloud_API_NSG funciona como um firewall: um conjunto de regras avaliadas por prioridade que permitem ou negam o tráfego de entrada. As regras criadas são: prioridade 100, SSH (porta 22) apenas de um endereço /32 do administrador; 110, HTTP (porta 80) de qualquer origem; e 120, HTTPS (porta 443) de qualquer origem. Todo o resto é negado pela regra padrão DenyAllInBound. A porta 8000 do Uvicorn não tem regra e portanto não é acessível de fora.

Custos. Geram cobrança a máquina virtual, o disco e o IP público; a rede virtual e o NSG não. Com a máquina ligada 24 horas, a estimativa com os preços de tabela consultados na API de preços da Azure é: VM cerca de US$ 6,9 por mês (US$ 0,0094 por hora), disco cerca de US$ 2,6 e IP cerca de US$ 3,7, totalizando cerca de US$ 13 por mês. Os preços do disco e do IP foram lidos em outra região e podem diferir um pouco; são preços de tabela, e não o consumo real dos créditos do Azure for Students, que deve ser consultado no portal. Desalocar a máquina interrompe a cobrança de computação, mas o disco e o IP continuam cobrando. O certificado HTTPS do Let's Encrypt é gratuito.

8. Implantação

O código é versionado no GitHub. Na máquina virtual, o script deploy/install.sh (idempotente) instalou o Python, o Nginx e as dependências; criou o usuário de serviço sem privilégios apitarefas, o ambiente virtual Python, o serviço systemd e a configuração do Nginx; e verificou as permissões. Em atualizações, o deploy/deploy.sh executa estes passos: (1) backup do SQLite, com abortamento se falhar; (2) git pull; (3) instalação das dependências; (4) atualização da unidade systemd; (5) sincronização da configuração do

Nginx, validada com nginx -t e restaurada automaticamente se houver erro; (6) reinício do serviço; e (7) verificação do /health.

O serviço api-tarefas executa o Uvicorn como usuário sem privilégios, com isolamento do systemd (ProtectSystem=strict, NoNewPrivileges, PrivateTmp) e permissão de escrita apenas em /var/lib/api-tarefas. O nome do serviço foi mantido da versão anterior para preservar a configuração.

HTTPS. O script deploy/https_ip.sh obtém um certificado do Let's Encrypt para o endereço IP público. Desde 2026 a autoridade emite certificados para IPs, com o perfil de curta duração (cerca de seis dias de validade) e o desafio HTTP-01, que exige o Certbot 5.4 ou superior. Os plugins nginx e apache do Certbot não instalam certificados de IP, então o script usa o modo certonly com --webroot, instala o bloco HTTPS do Nginx e registra um *hook* que recarrega o Nginx após cada renovação. A emissão foi testada primeiro no ambiente de testes (*staging*) e depois feita no ambiente real. A renovação automática é feita por um timer do systemd, que roda duas vezes ao dia. Foram verificados: a cadeia de confiança pelo curl e pelo Chrome, a presença do IP no certificado, a simulação de renovação e uma renovação forçada real, após a qual o Nginx passou a servir o novo certificado (número de série diferente) sem intervenção manual.

Decidiu-se **não** redirecionar o HTTP para o HTTPS e **não** habilitar HSTS. Como o certificado dura poucos dias e a máquina pode ficar desligada para economizar créditos, o HTTP funciona como contingência; o HSTS prenderia os navegadores ao HTTPS e tornaria uma falha de renovação mais grave.

O Diagrama 5 resume o fluxo de implantação, e o procedimento completo, incluindo o *rollback*, está em docs/DEPLOY_AZURE_REAL.md.

9. Segurança

Controles implementados.

- **Rede.** O NSG libera apenas as portas 80 e 443 ao público e o SSH somente ao IP do administrador. A porta 8000 não é exposta (Uvicorn só em 127.0.0.1). O acesso à máquina é por chave SSH, sem senha.
- **Escrita restrita.** A API não possui autenticação. Para compensar, o Nginx libera leitura (GET) ao público e permite POST, PUT, PATCH e DELETE apenas aos IPs listados em um arquivo local da máquina, fora do GitHub; os demais recebem HTTP 403. A regra vale nas portas 80 e 443.
- **HTTPS.** Tráfego criptografado, com TLS 1.2 e 1.3, certificado válido emitido pelo Let's Encrypt e renovação automática.
- **Aplicação.** Consultas parametrizadas (ORM), validação rigorosa de entrada, painel sem interpretação de HTML (XSS) e com CSP, sem CORS aberto, erros sem detalhes internos.
- **Sistema.** Serviço executado por usuário sem privilégios, com isolamento do systemd; banco com permissão 600; arquivo .env com permissão 600 e dono root.
- **Segredos e dependências.** Chaves, .env, bancos e backups estão no .gitignore e não são publicados. As dependências são fixadas e auditadas com pip-audit.

Sobre autenticação. Avaliou-se se a restrição por IP bastaria para este trabalho. Como os dados são fictícios, a leitura é pública por desenho e a escrita fica restrita, concluiu-se que

ela é suficiente para a demonstração. Uma autenticação improvisada (senhas ou tokens mal armazenados) seria pior do que nenhuma. O caminho recomendado para evolução seria exigir um token de API, guardado no .env e comparado em tempo constante, sempre sobre HTTPS.

Limitações. A restrição por IP depende de o IP do administrador ser estável; não há autenticação de usuários; os backups ficam no mesmo disco da máquina; e a máquina é única, sem redundância.

10. Testes

Testes automatizados locais. A suíte tem **96 testes aprovados, sem avisos**, executados com banco SQLite temporário e isolado: 55 verificam a aplicação e o instalador, e 41 verificam a reprodutibilidade e a configuração de entrega. Cobrem inicialização e health check, CRUD, paginação e filtros, fluxo e datas de status, validações (severidade, status, categoria, IP e campos obrigatórios), estatísticas com banco vazio e preenchido, persistência após reabrir o banco, preservação da tabela antiga, restrições do banco, OpenAPI, dashboard e arquivos estáticos, ausência de CDN, XSS, injeção de SQL, erro interno sem vazamento de detalhes, CORS fechado, presença e integridade dos arquivos do Swagger, sintaxe dos scripts, configuração do Nginx (proteção de escrita, ACME, TLS, ausência de HSTS e de redirecionamento forçado) e do systemd.

Integração contínua. A cada envio ao GitHub, o GitHub Actions cria um ambiente limpo, instala as dependências, verifica a presença dos arquivos necessários e executa a suíte. Não há credenciais da Azure nem implantação automática no fluxo.

Testes na URL pública. O script scripts/testar_producao.sh executou **58 verificações, todas aprovadas, com código de saída 0**, incluindo HTTPS exigido: disponibilidade e interface; arquivos do Swagger; coerência das estatísticas com a listagem; CRUD completo e transições de status; validações (422) e recurso inexistente (404); reinício do serviço e persistência; porta 8000 fechada; bloqueio de escrita por IP não autorizado (HTTP 403 nos quatro métodos, em HTTP e HTTPS); permissões do banco; certificado válido, emitido pelo Let's Encrypt e com o IP; timer e hook de renovação; e integridade do SQLite, com a existência das duas tabelas verificada em sqlite_master. Um teste negativo (chave SSH inválida de propósito) confirmou que o script contabiliza 13 falhas e encerra com código diferente de zero, e que remove o incidente de teste mesmo assim. Os logs ficam em docs/evidencias, e o log da versão anterior do script foi mantido para histórico.

11. Evidências

As figuras a seguir foram obtidas da aplicação e da infraestrutura reais. As Figuras 1 a 6 são telas do Portal Azure; as Figuras 7 a 13 e 17 são páginas abertas no navegador (a faixa escura superior, com URL e horário, foi acrescentada à captura); as Figuras 14, 15, 16 e 18 são janelas reais de console. Nas imagens, o endereço IP do administrador, o identificador da assinatura e o e-mail da conta estão mascarados.

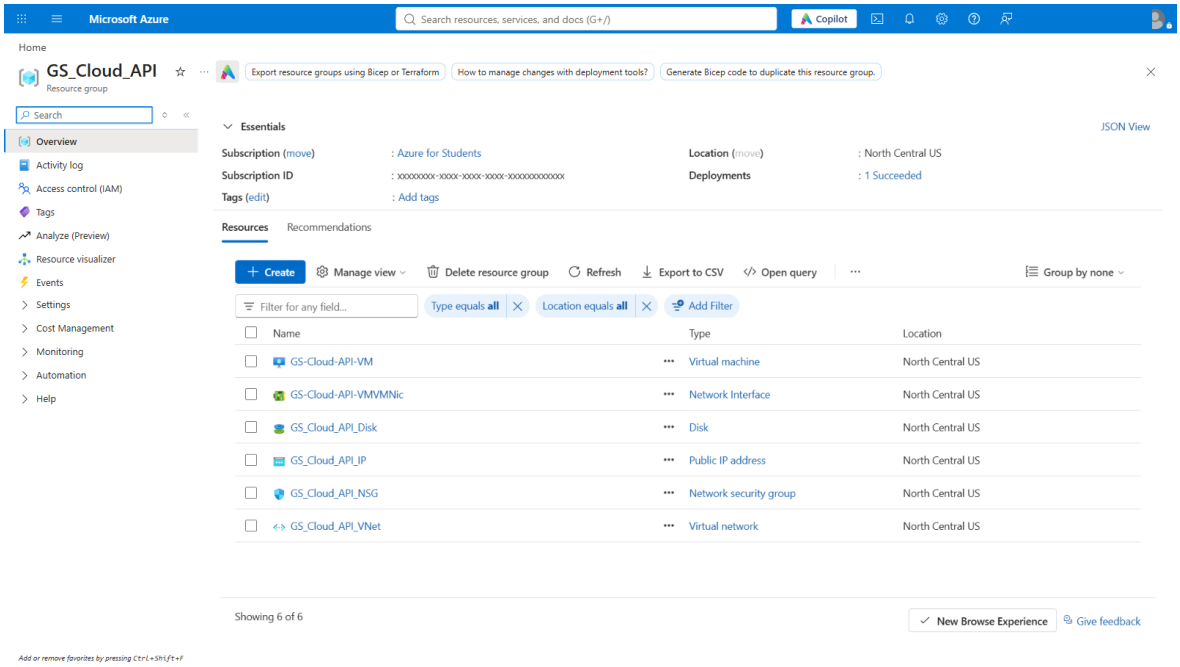


Figura 1 — Grupo de recursos utilizado para organizar os componentes da aplicação hospedada na Microsoft Azure.

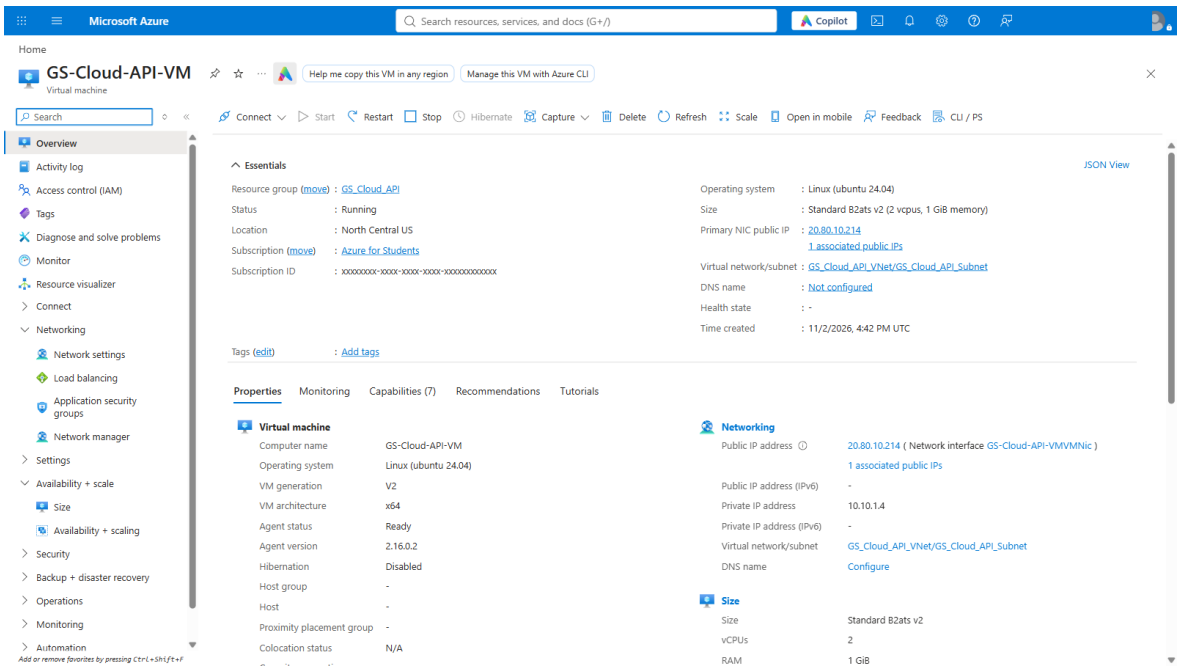


Figura 2 — Máquina virtual utilizada para executar a API REST na Microsoft Azure.

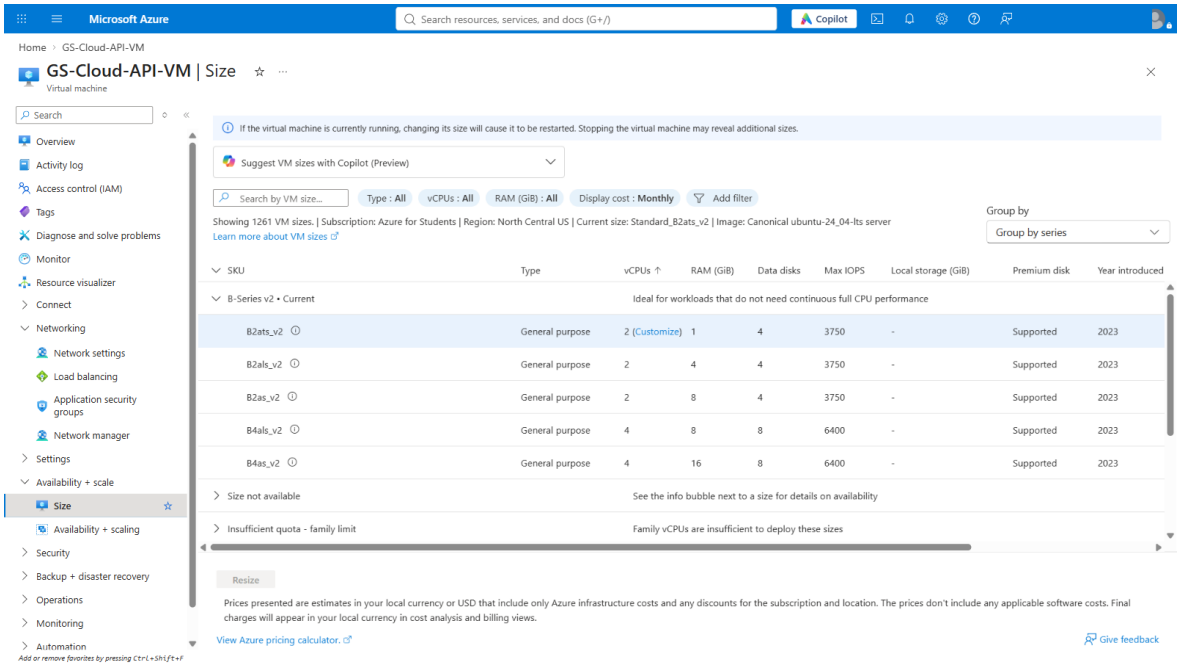


Figura 3 — Configuração de processamento e memória da máquina virtual utilizada no projeto.

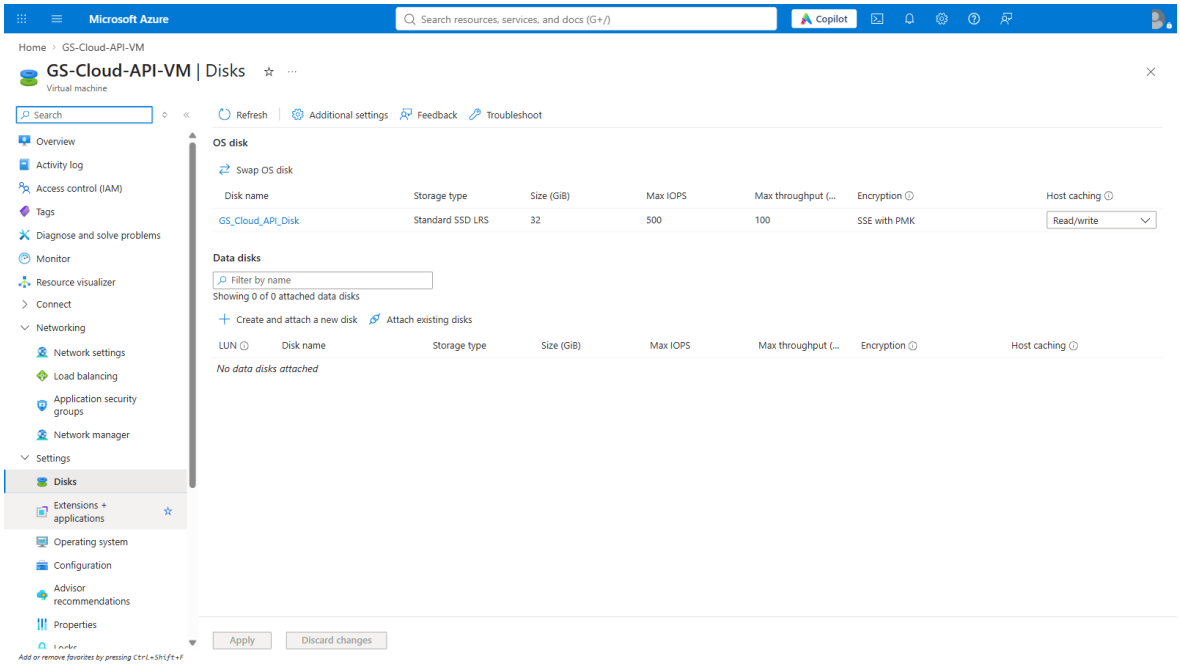


Figura 4 — Armazenamento persistente disponibilizado por meio dos discos gerenciados da Microsoft Azure.

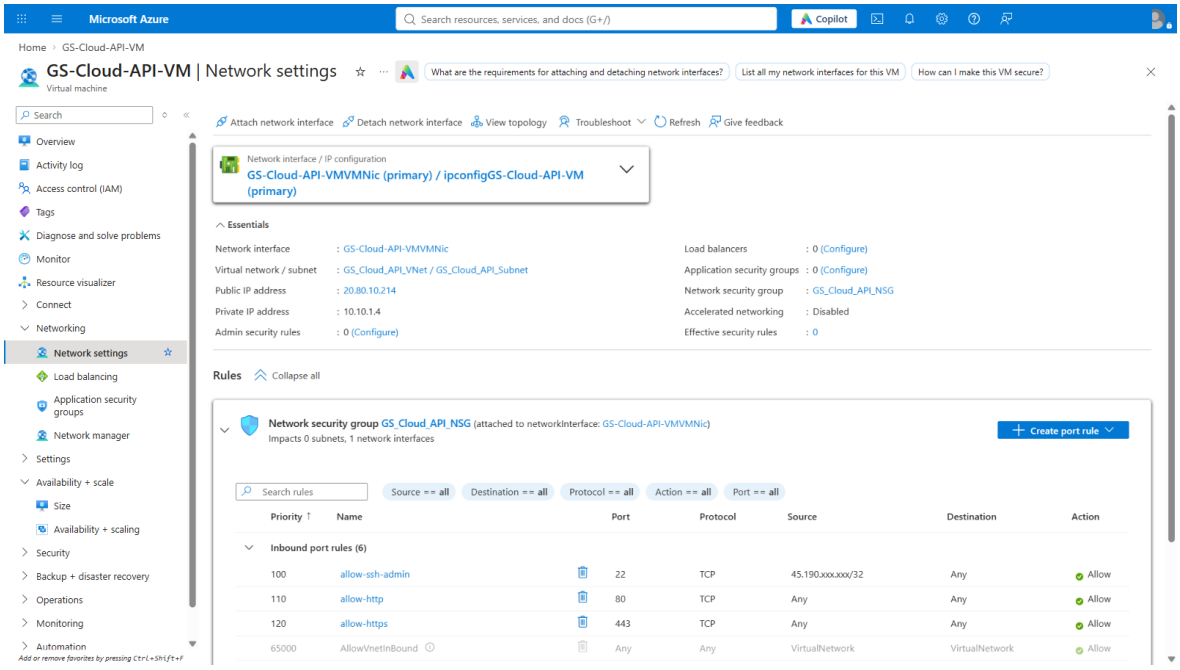


Figura 5 — Infraestrutura de rede responsável pela comunicação da máquina virtual com os demais componentes.

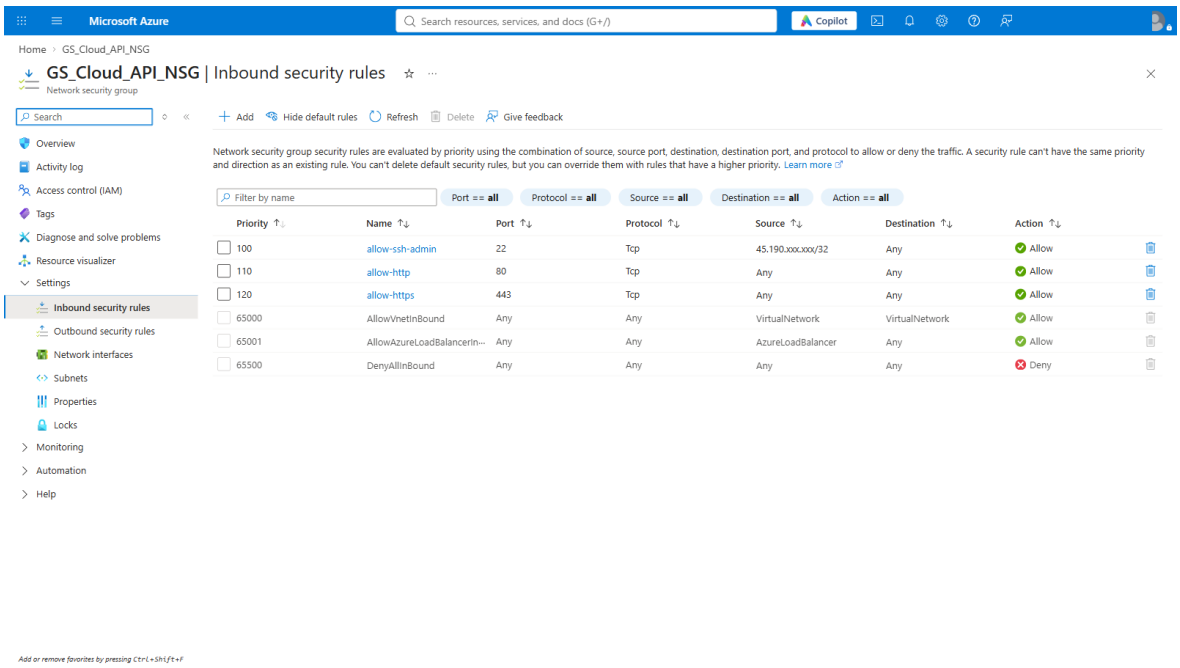


Figura 6 — Regras de segurança de rede utilizadas para controlar o acesso à máquina virtual.

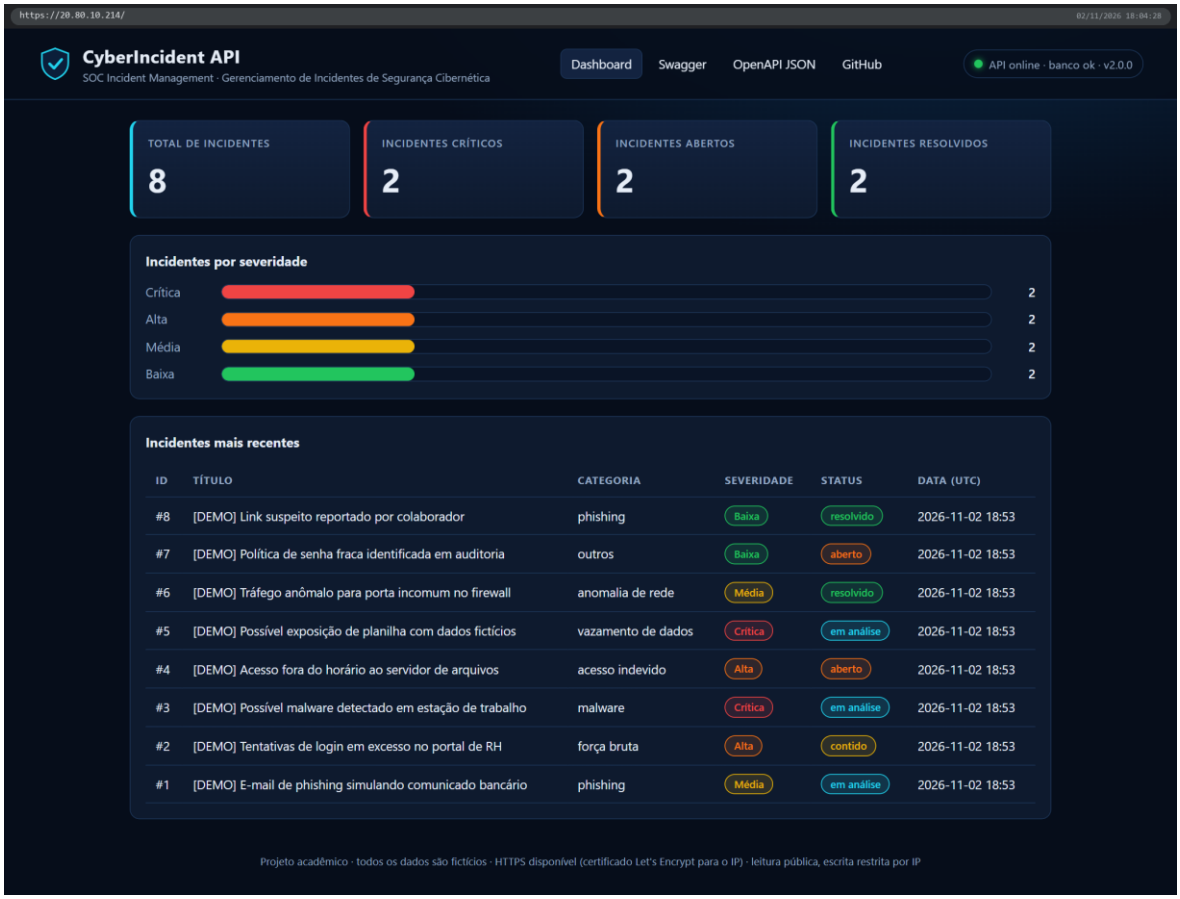


Figura 7 — Dashboard da CyberIncident API, acessado por HTTPS, com indicadores, gráfico por severidade e tabela de incidentes.

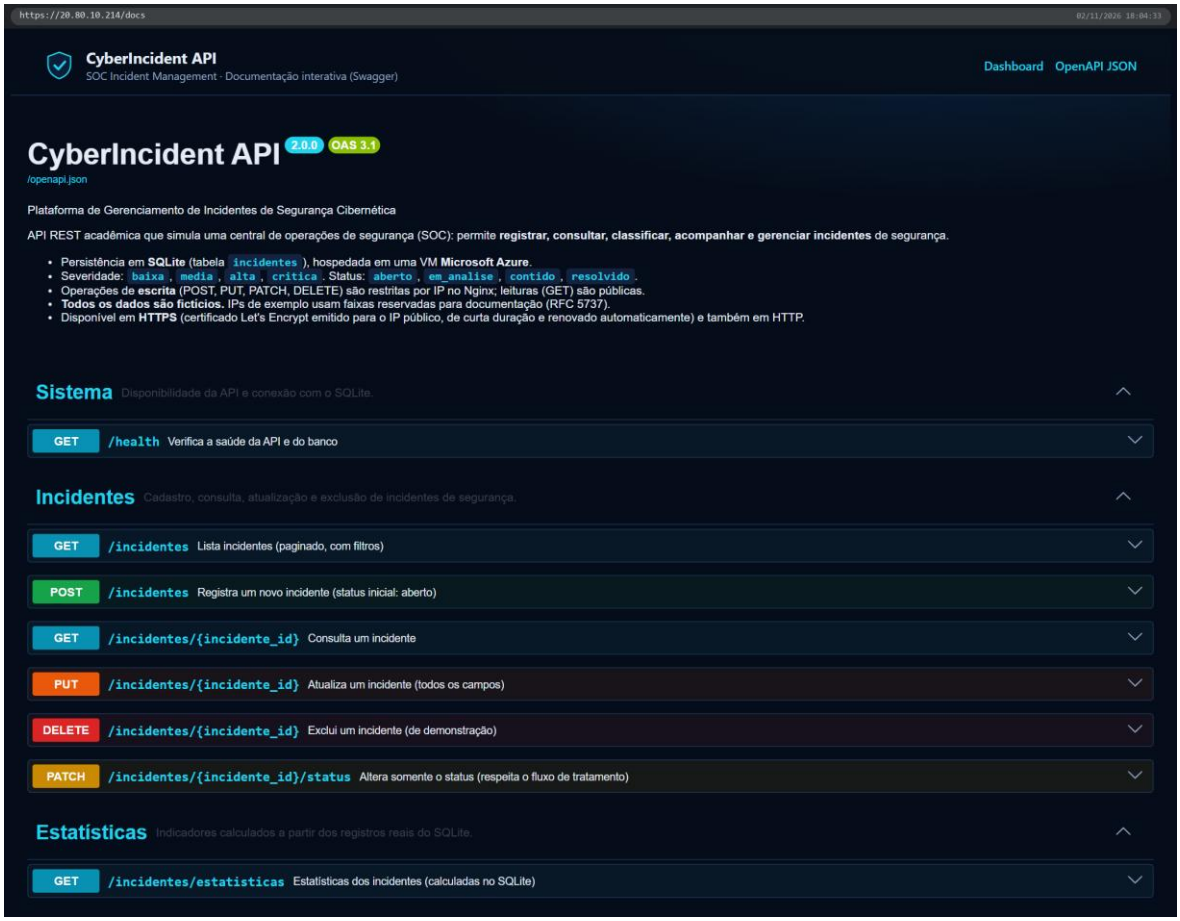


Figura 8 — Documentação Swagger/OpenAPI personalizada da aplicação publicada na Microsoft Azure.

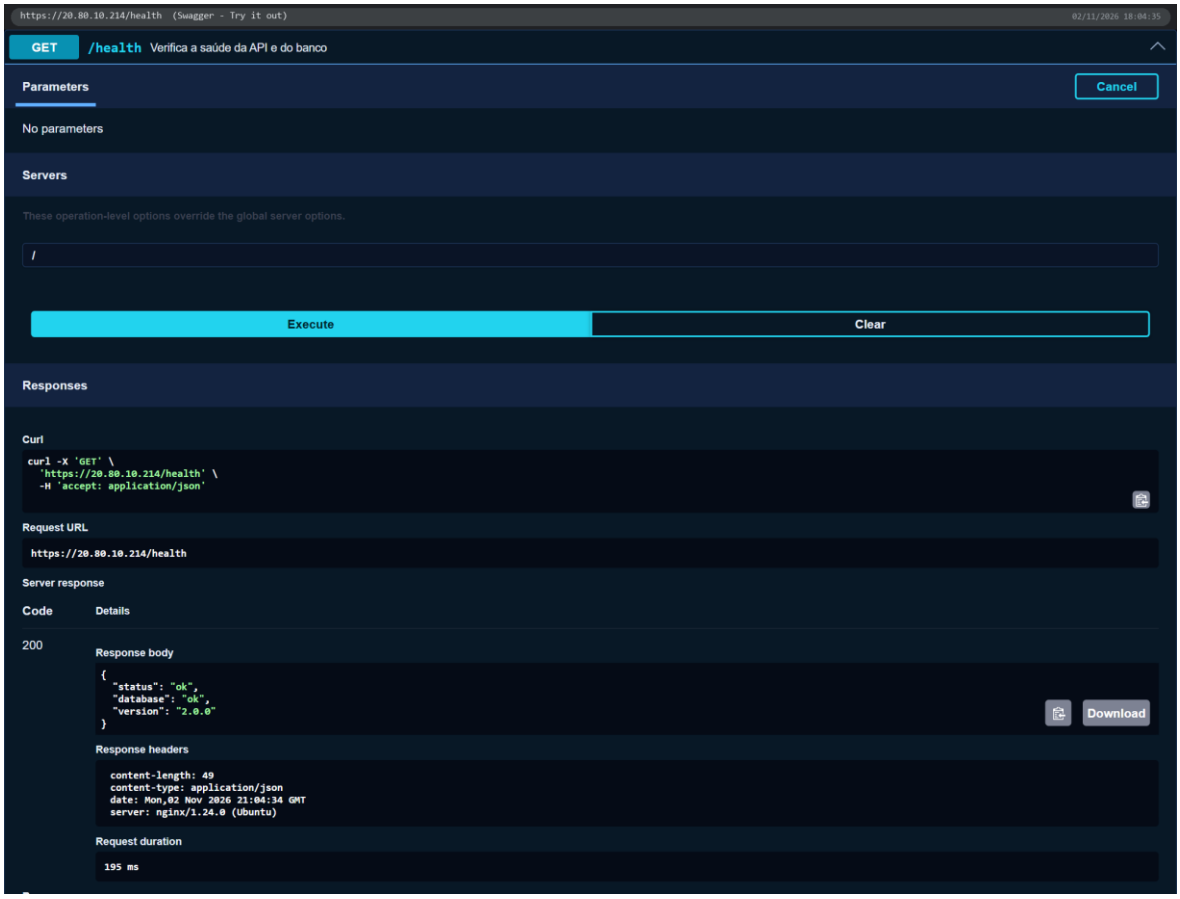


Figura 9 — Verificação da disponibilidade da API e da conexão com o banco SQLite (HTTP 200).

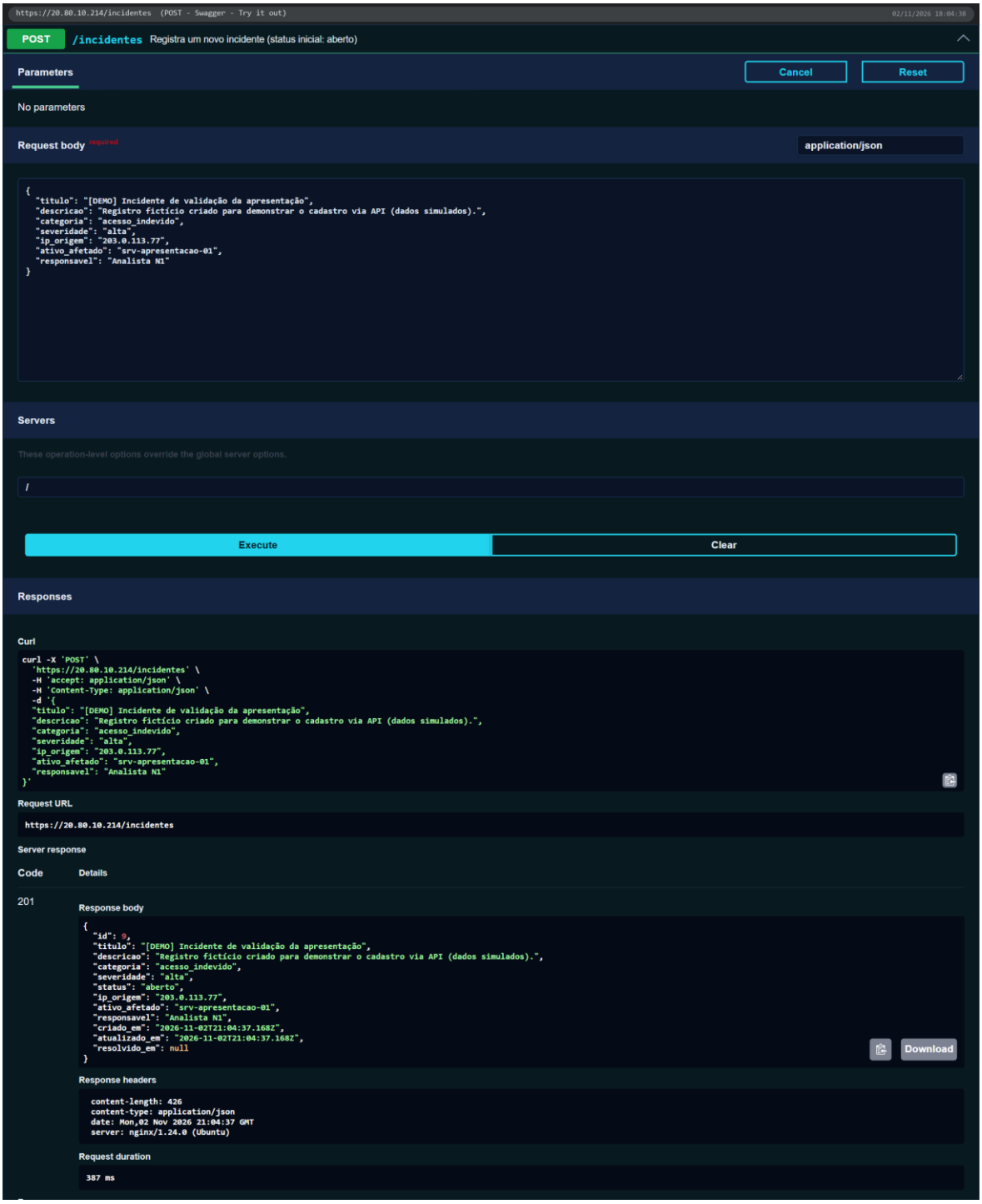


Figura 10 — Cadastro de incidente por meio de requisição HTTP POST (HTTP 201).

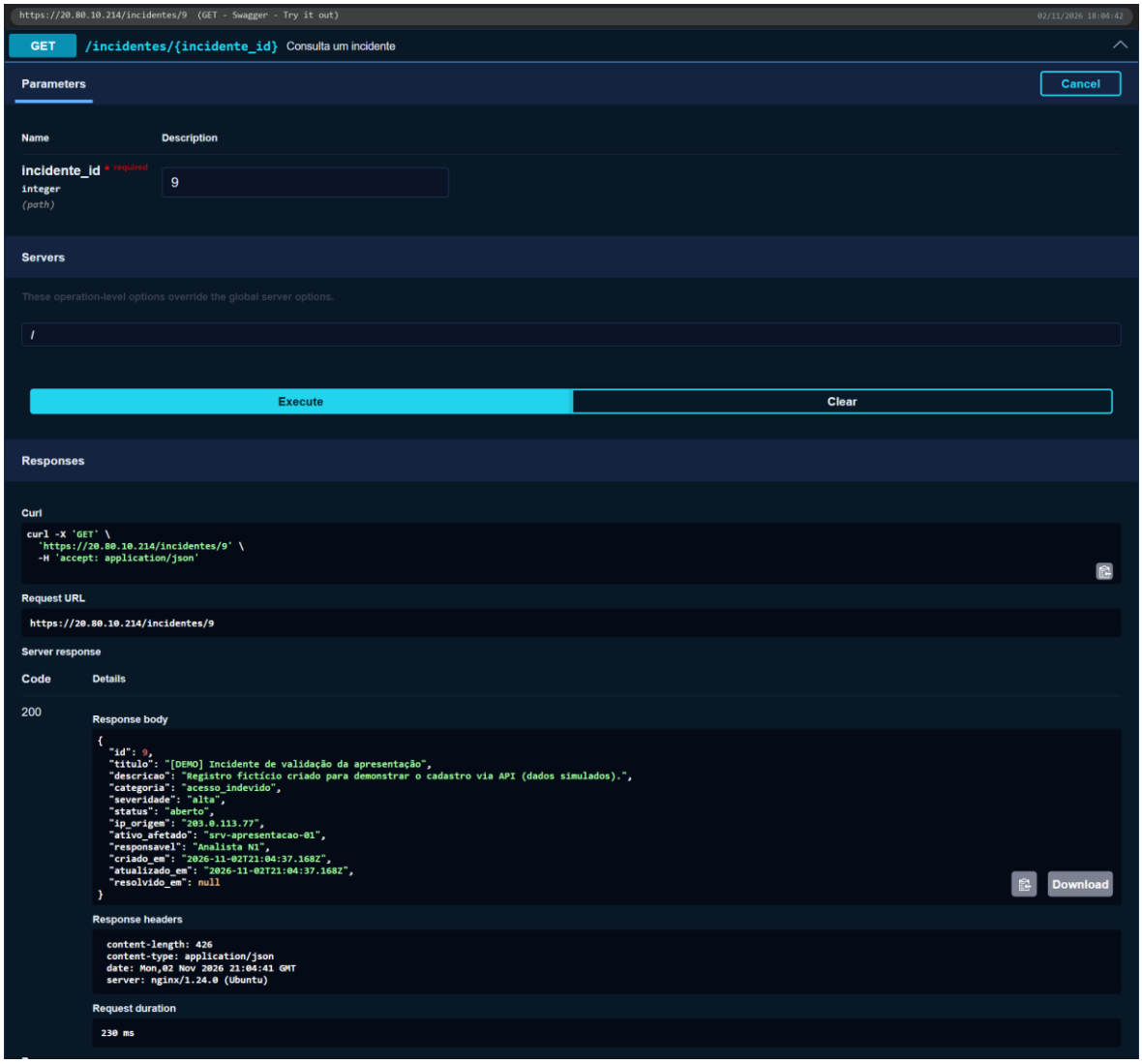


Figura 11 — Consulta de incidente persistido no SQLite por meio de requisição HTTP GET (HTTP 200).

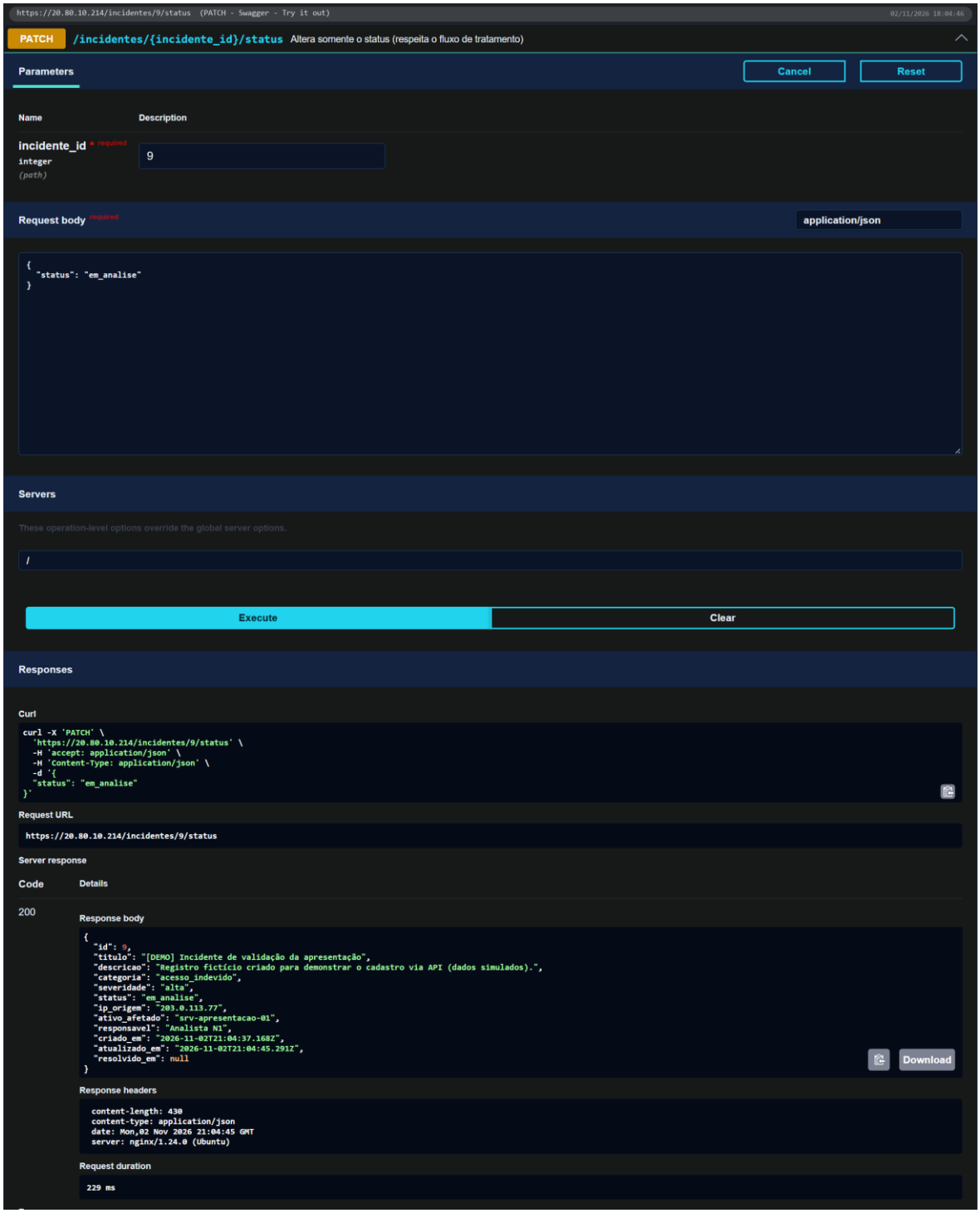


Figura 12 — Alteração de status do incidente por meio de requisição HTTP PATCH.

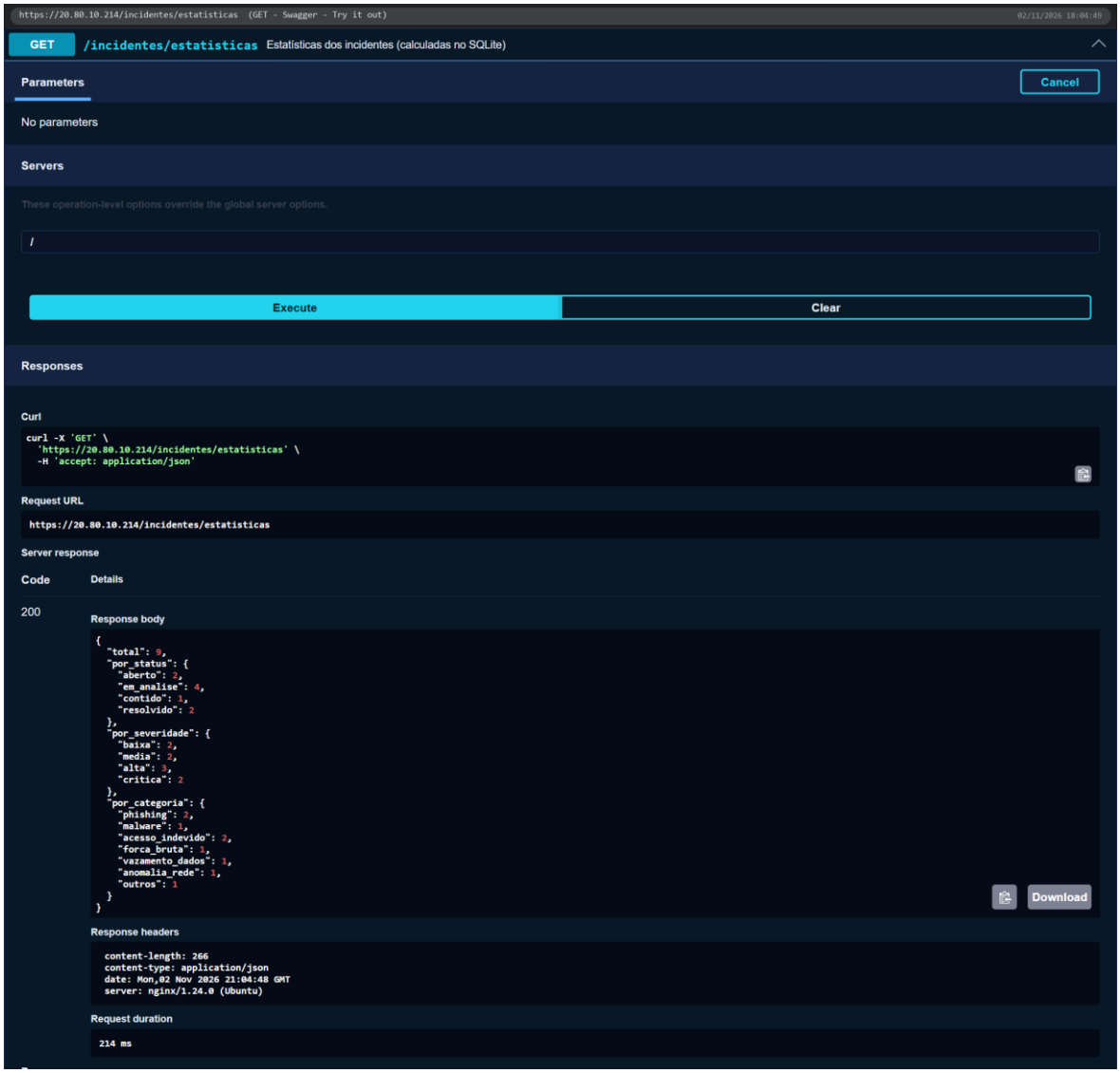


Figura 13 — Estatísticas calculadas a partir dos registros do SQLite.

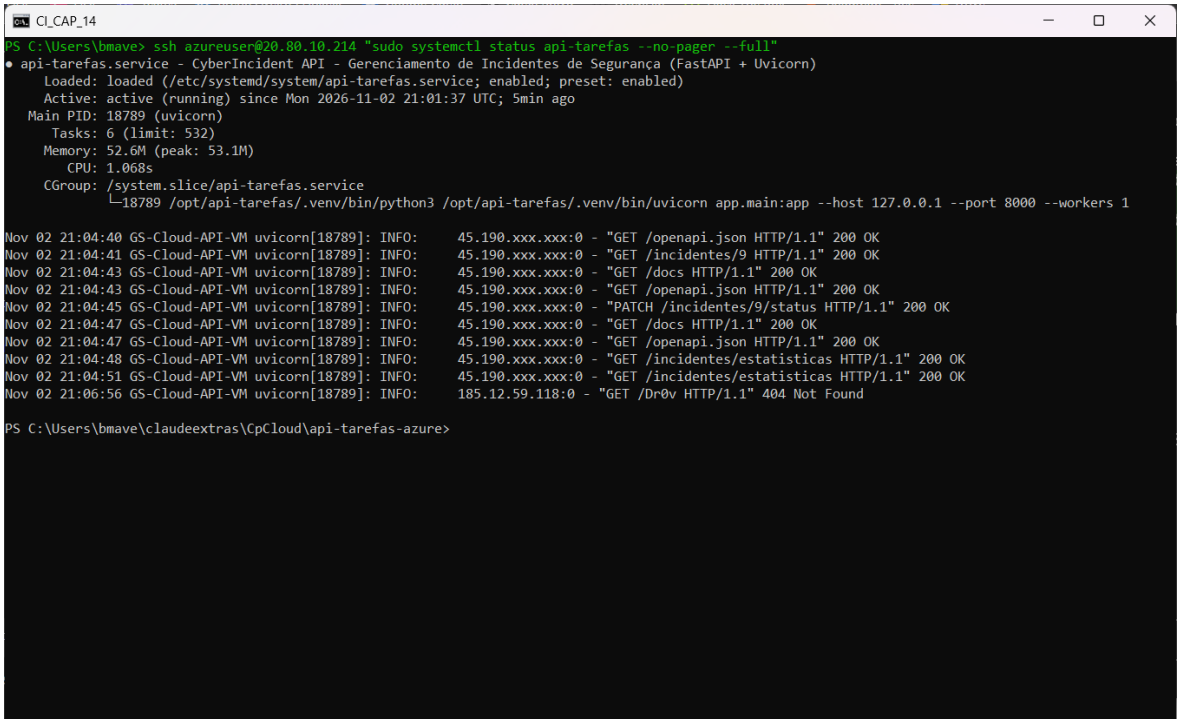


Figura 14 — Serviço systemd responsável pela execução contínua da API na máquina virtual Linux.

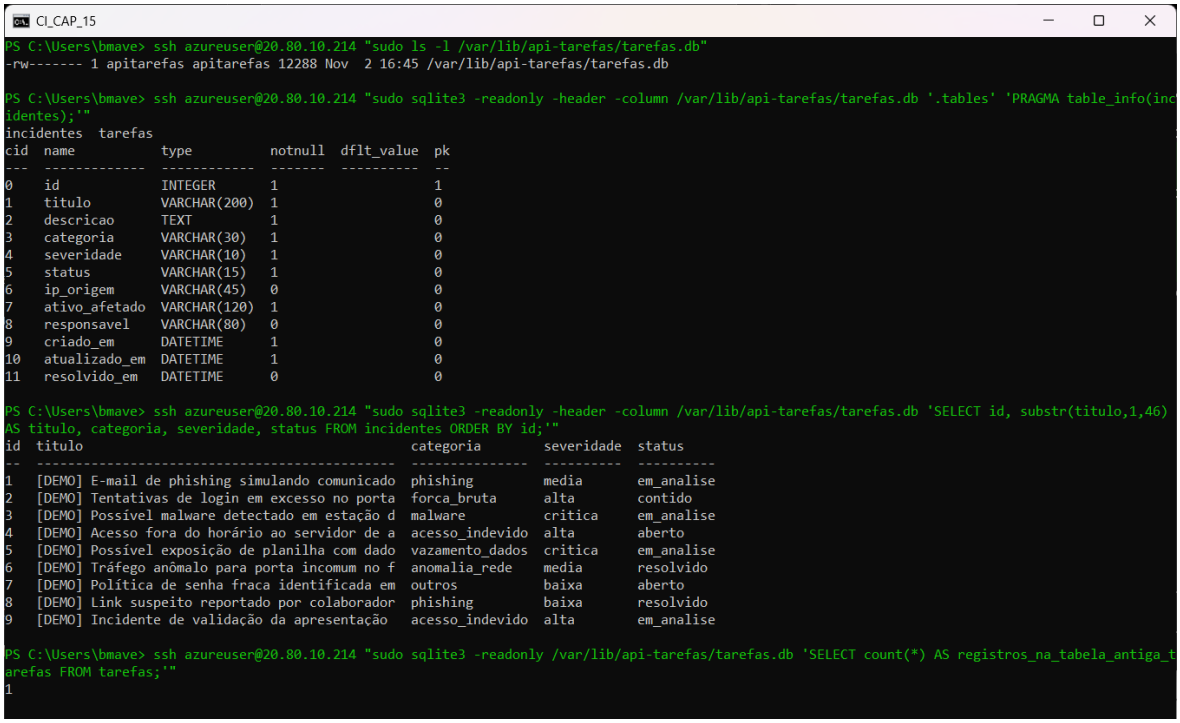


Figura 15 — Consulta direta ao banco SQLite armazenado no disco persistente da máquina virtual.

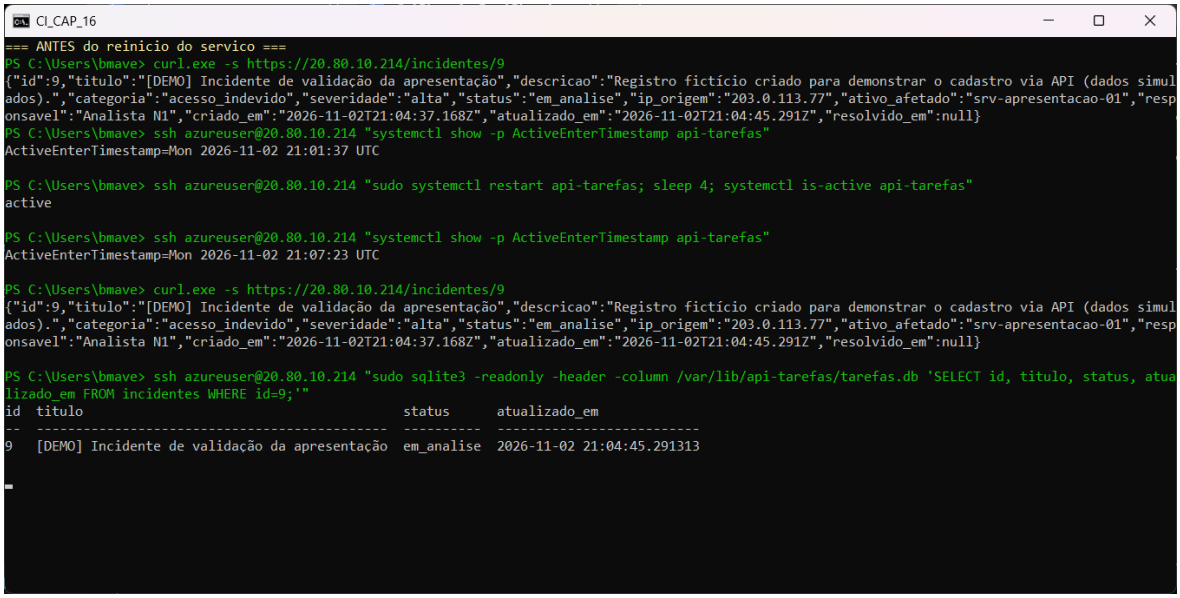


Figura 16 — Persistência dos registros SQLite após a reinicialização do serviço da aplicação.

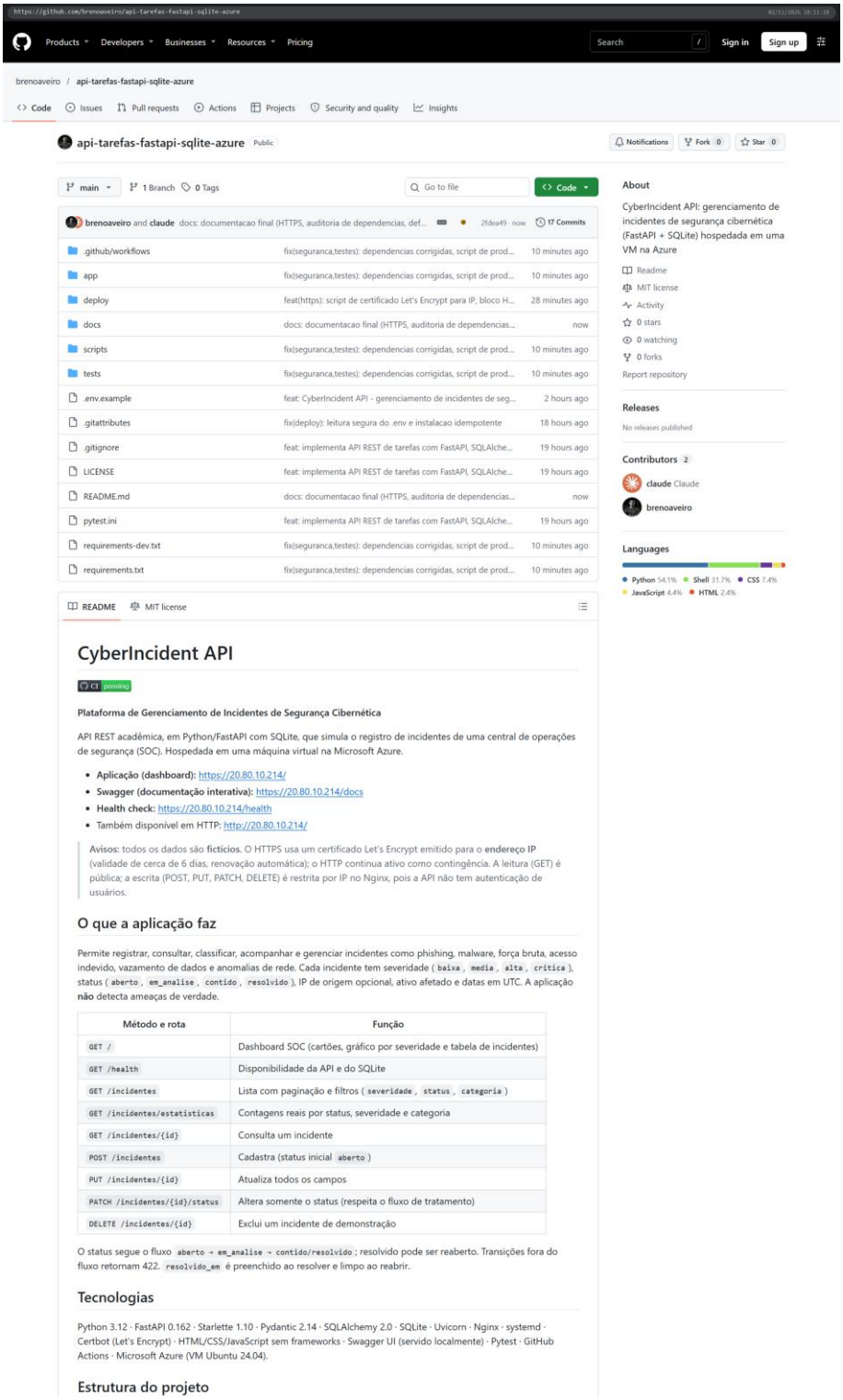


Figura 17 — Repositório GitHub utilizado para armazenar e versionar o código-fonte da aplicação.

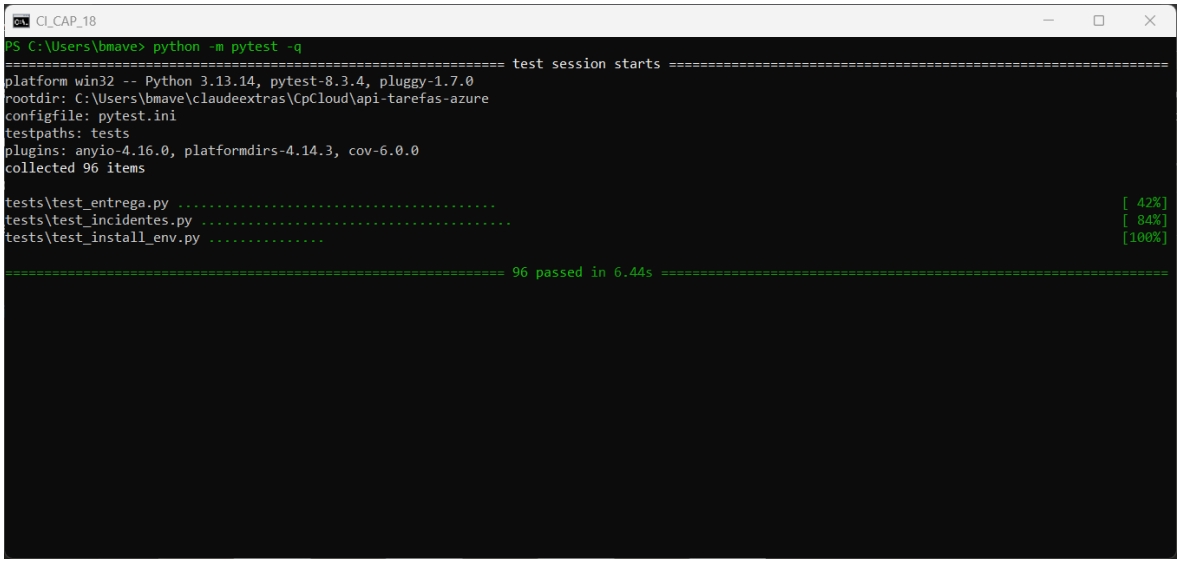


Figura 18 — Execução dos testes automatizados utilizados para validar o funcionamento da API.

12. Diagramas

Os seis diagramas foram escritos em Mermaid, a partir do código e da infraestrutura reais, e estão em docs/diagramas. O Diagrama 1 mostra a arquitetura na Azure, do navegador ao SQLite no disco gerenciado, com o NSG, a rede virtual e a máquina virtual. O Diagrama 2 detalha o fluxo HTTP e HTTPS entre o navegador, o Nginx, o FastAPI e o SQLite, incluindo o bloqueio de escrita. O Diagrama 3 apresenta o modelo de dados da tabela incidentes. O Diagrama 4 mostra o cadastro, a validação e o fluxo de status. O Diagrama 5 representa o caminho do código do GitHub até a máquina virtual, e o Diagrama 6, o fluxo de testes e validação.

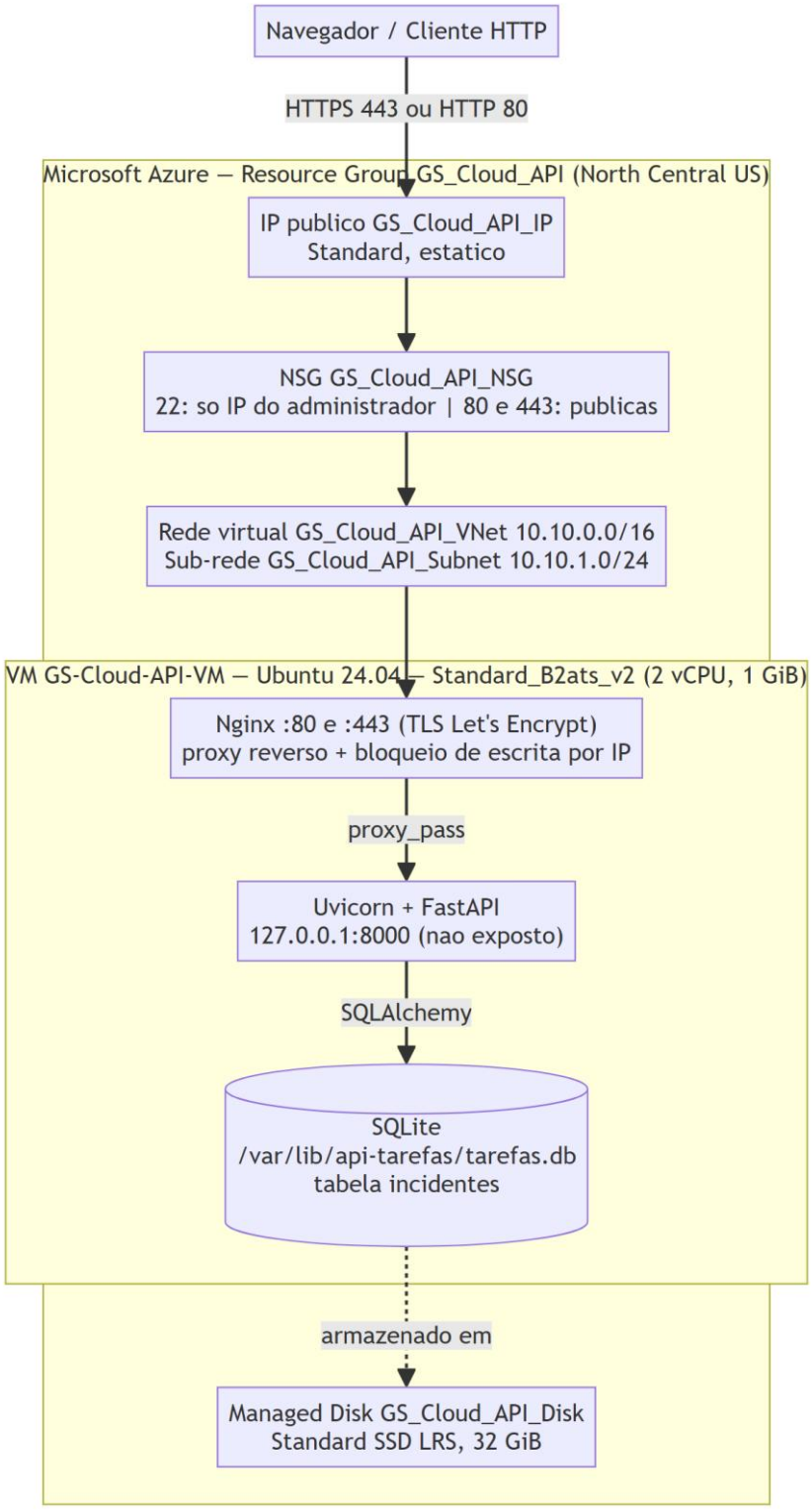


Diagrama 1 — Arquitetura da CyberIncident API na Microsoft Azure.

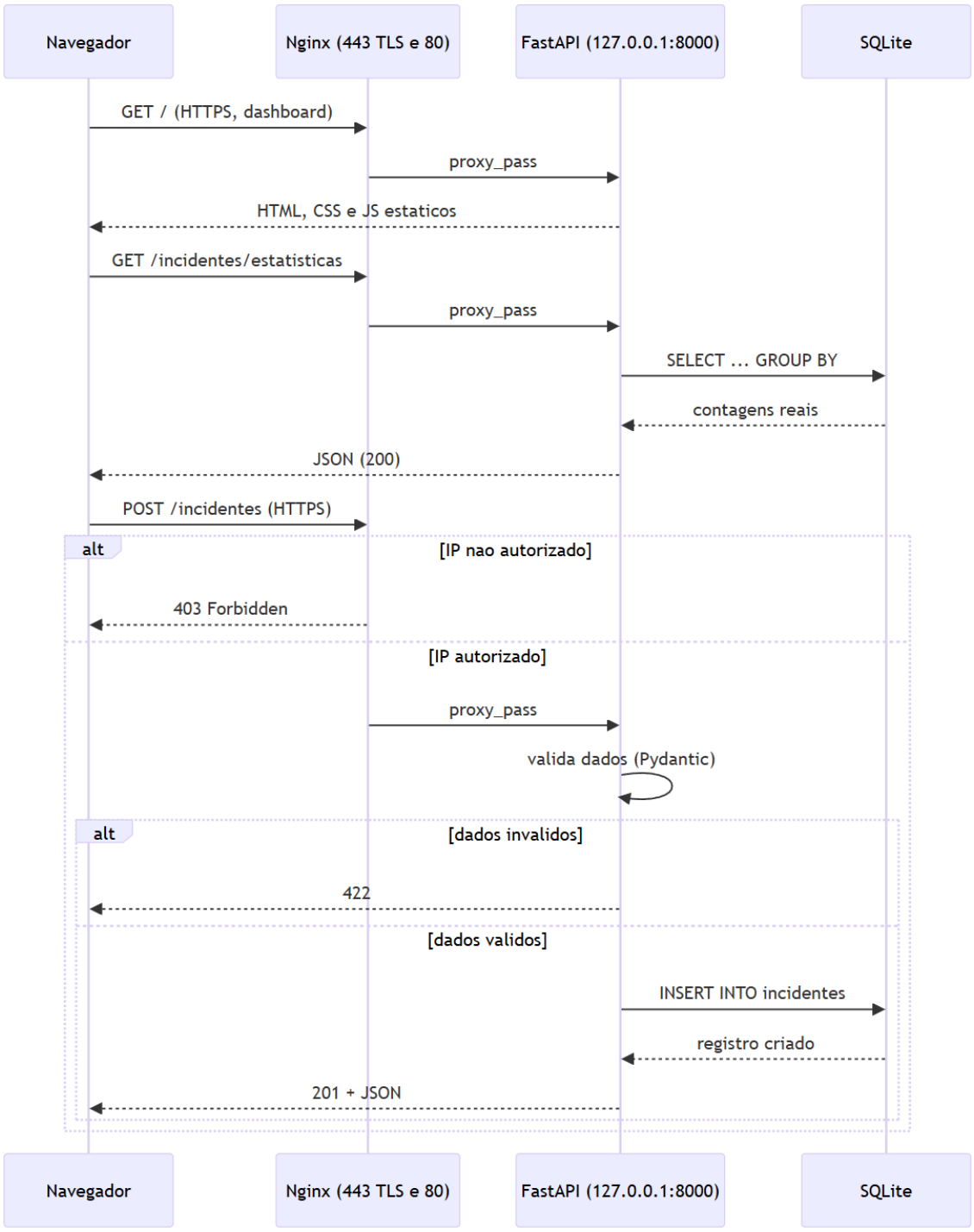


Diagrama 2 — Fluxo das requisições entre o navegador, o Nginx, o FastAPI e o SQLite.

INCIDENTES			
INTEGER	id	PK	
VARCHAR	titulo		
TEXT	descricao		
VARCHAR	categoria		phishing, malware, acesso_indevido, forca_bruta, vazamento_dados, anomalia_rede, outros
VARCHAR	severidade		baixa, media, alta, critica
VARCHAR	status		aberto, em_analise, contido, resolvido
VARCHAR	ip_origem		opcional, IPv4 ou IPv6
VARCHAR	ativo_afetado		
VARCHAR	responsavel		opcional
DATETIME	criado_em		UTC
DATETIME	atualizado_em		UTC
DATETIME	resolvido_em		preenchido ao resolver

Diagrama 3 — Modelo de dados da tabela incidentes.

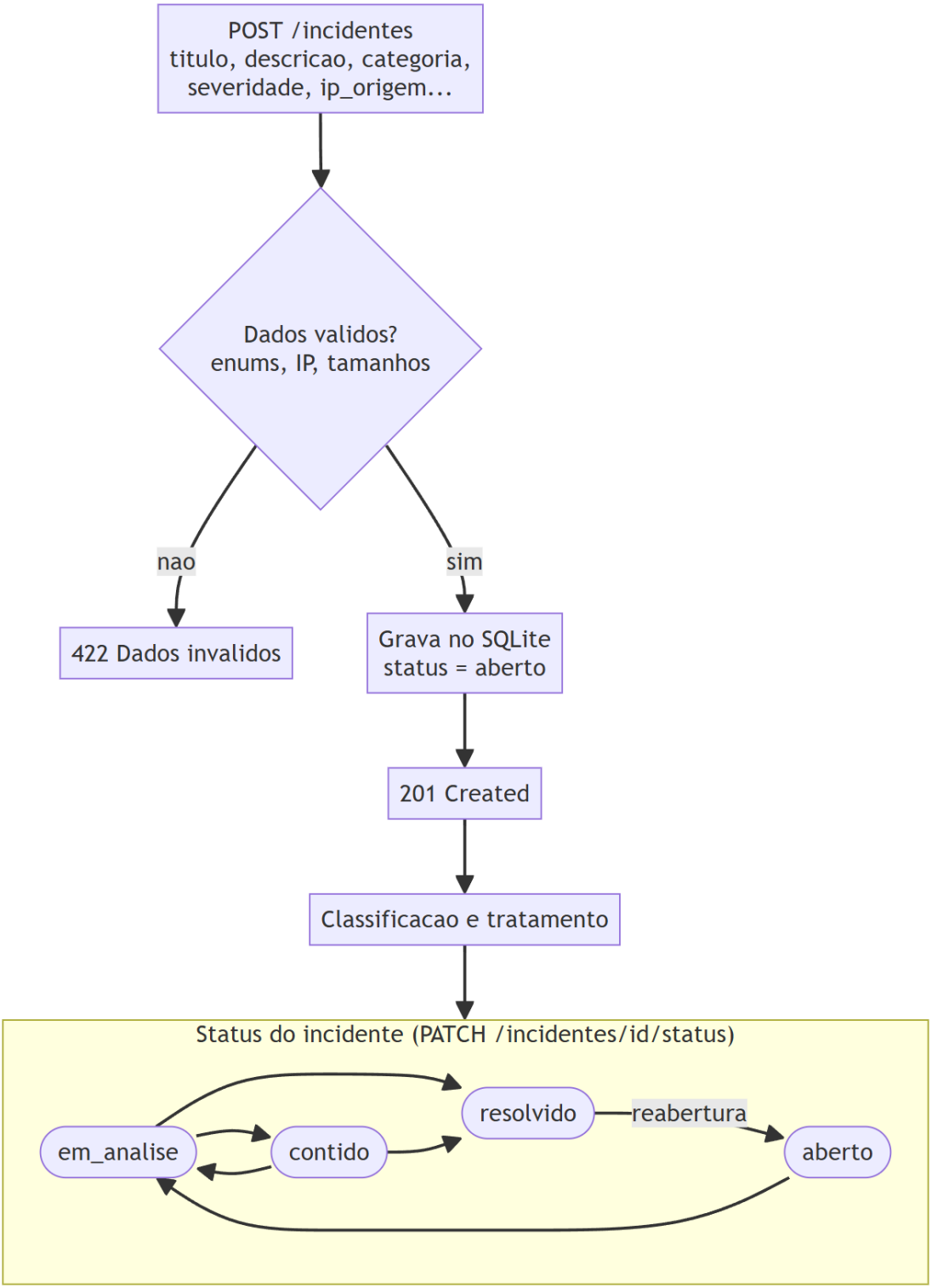


Diagrama 4 — Cadastro e classificação de incidentes e fluxo de status.

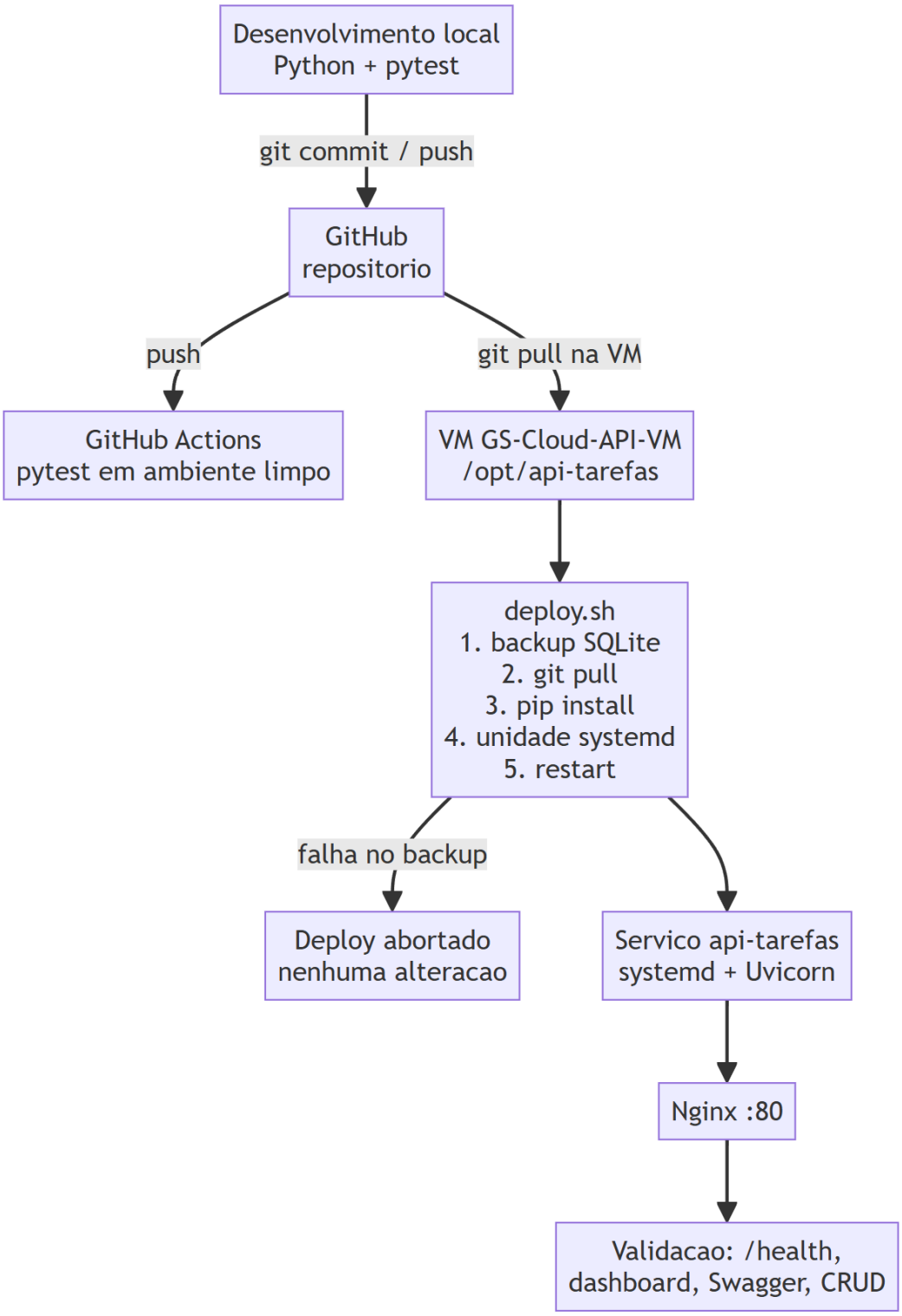


Diagrama 5 — Fluxo de implantação do GitHub para a Azure.

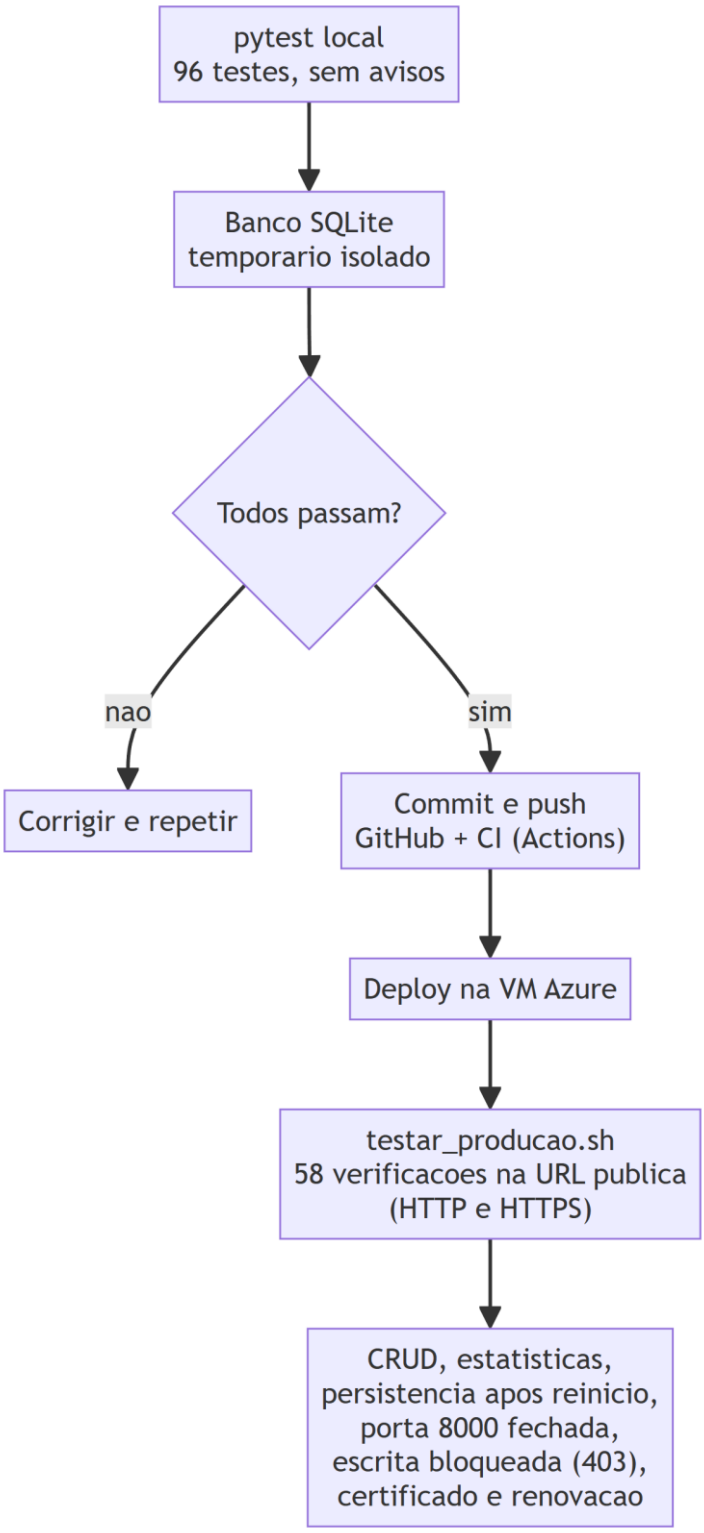


Diagrama 6 — Fluxo de testes e validação.

13. Custos

O quadro resume as premissas e os valores estimados com a máquina ligada 24 horas por dia, em Linux e sob demanda. São preços de tabela e devem ser confirmados na calculadora oficial; não representam o consumo efetivo dos créditos do Azure for Students.

Recurso	Preço de tabela	Estimativa mensal
VM Standard_B2ats_v2 (North Central US)	US$ 0,0094 por hora	cerca de US$ 6,9
Disco Standard SSD de 32 GiB	cerca de US$ 2,64 por mês	cerca de US$ 2,6
IP público Standard	cerca de US$ 0,005 por hora	cerca de US$ 3,7
Certificado Let's Encrypt	gratuito	US$ 0
Total aproximado		**cerca de US$ 13 por mês**

Rede virtual e NSG não têm custo direto. Desalocar a máquina virtual interrompe a cobrança de computação; o disco e o IP continuam cobrando. O saldo de créditos não é exibido pela linha de comando e deve ser conferido no Portal Azure.

14. Limitações e melhorias possíveis

- **Sem autenticação de usuários.** A escrita depende da restrição por IP no Nginx. Melhoria: token de API ou provedor de identidade, sempre sobre HTTPS.
- **Certificado de curta duração.** Se a máquina permanecer desligada por mais de seis dias, o certificado expira e é renovado após a próxima inicialização; nesse intervalo o HTTPS fica inválido e o HTTP continua funcionando. Melhoria: domínio próprio com certificado de 90 dias.
- **SQLite.** Não é indicado para muita escrita simultânea nem oferece replicação. Melhoria: banco gerenciado.
- **Máquina única e 1 GiB de RAM.** Sem alta disponibilidade, com uso de *swap*. Melhoria: tamanho maior e mais de uma instância.
- **Backups no mesmo disco.** Uma falha do disco afetaria o banco e as cópias. Melhoria: cópias em armazenamento separado.
- **Evidências do Portal.** As capturas do Portal Azure dependem de login manual e não são reproduzidas pelo pipeline de CI.

15. Conclusão

A CyberIncident API foi desenvolvida, publicada e verificada de ponta a ponta. Os resultados comprovados foram: aplicação funcional com dashboard e Swagger próprios; persistência confirmada após o reinício do serviço, com a tabela antiga preservada; 96 testes automatizados locais e integração contínua aprovados; 58 verificações aprovadas na URL pública; HTTPS com certificado válido e renovação automática comprovada; dependências sem vulnerabilidades conhecidas; e a proteção de escrita por IP mantida nas duas portas.

O trabalho permitiu aplicar conceitos centrais de computação em nuvem — máquina virtual, SKU, rede, firewall, disco persistente, implantação e HTTPS — e de desenvolvimento de APIs: modelagem, validação, documentação, testes e segurança. As limitações restantes estão registradas, com caminhos para superá-las.

16. Referências

- FastAPI. Documentação oficial. https://fastapi.tiangolo.com
- Pydantic. Documentação oficial. https://docs.pydantic.dev
- SQLAlchemy. Documentação oficial. https://docs.sqlalchemy.org

- SQLite. Documentação oficial, incluindo o modo WAL e a API de backup. https://www.sqlite.org/docs.html
- Uvicorn. Documentação oficial. https://www.uvicorn.org
- Nginx. Documentação oficial. https://nginx.org/en/docs/
- systemd. Manual de referência. https://www.freedesktop.org/software/systemd/man/
- Microsoft. Tamanhos de máquinas virtuais no Azure. https://learn.microsoft.com/azure/virtual-machines/sizes
- Microsoft. Visão geral dos discos gerenciados do Azure. https://learn.microsoft.com/azure/virtual-machines/managed-disks-overview
- Microsoft. Grupos de segurança de rede do Azure. https://learn.microsoft.com/azure/virtual-network/network-security-groups-overview
- Microsoft. Calculadora de preços do Azure. https://azure.microsoft.com/pricing/calculator/
- Let's Encrypt. Certificados de 6 dias e para endereços IP — disponibilidade geral. https://letsencrypt.org/2026/01/15/6day-and-ip-general-availability
- Let's Encrypt. Certificados de curta duração e para IP no Certbot. https://letsencrypt.org/2026/03/11/shorter-certs-certbot
- Certbot. Guia do usuário. https://eff-certbot.readthedocs.io/en/stable/using.html
- GitHub. Documentação do GitHub Actions. https://docs.github.com/actions
- PyPA. Advisory Database e pip-audit. https://github.com/pypa/pip-audit
- IETF. RFC 5737 — Blocos de endereços IPv4 reservados para documentação. https://www.rfc-editor.org/rfc/rfc5737
- Swagger UI. https://swagger.io/tools/swagger-ui/